TrafficSense: Urban Traffic Flow Analytics & Congestion Prediction

Big Data Analytics Project

Department of Computer Science and Engineering
College of Engineering, Cherthala

Faculty Guide: Mrs.Shefna Naushad

Academic Year 2026–2027

Project Team

Abhijeet Rameshkumar Nair (CEC24AD005) Ananthukrishna CU (CEC24AD014)
Amarnath A (LCEC24AD061) Abhinav KP (LCEC24AD058)

Abstract

Problem

Traffic volume varies over time and weather. This project analyzes historical patterns, not live feeds.

Visualization

Hive/Spark summaries, Python plots, and Power BI communicate findings.

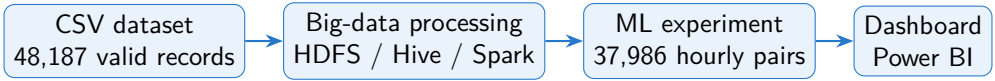
Approach

Python, Hadoop/HDFS, Hive, Spark/PySpark, and MLlib analyze Metro Interstate traffic records.

Key result

93.18%

chronological test accuracy for separate next-hour congestion prediction.



The experiment used 30,388 training and 7,598 test examples.

Introduction: Why Traffic Analytics Matters

Background

- Urban traffic patterns vary throughout the day.
- Weather and calendar conditions can coincide with changes in volume.
- Historical records reveal recurring peak periods and congestion distributions.
- Big Data tools support systematic organization, querying, and analysis.

Problem statement

Historical traffic records can be difficult to inspect manually. Aggregating patterns across time and weather requires a systematic workflow, while descriptive analytics alone does not test next-hour prediction. Model outputs also need validation before operational use.

Research question

How can historical traffic data be processed and analyzed to understand congestion patterns and explore next-hour prediction?

Objectives and Project Scope

Overall objective

Develop an integrated Big Data Analytics workflow for historical traffic analysis and congestion-prediction experimentation.

Objectives

1. Clean and prepare the dataset.
2. Analyze time and weather patterns.
3. Use HDFS and Hive for storage and queries.
4. Run Spark processing and ML experiments.
5. Evaluate next-hour prediction chronologically.
6. Communicate findings with charts and Power BI.

In scope

Historical traffic, hourly/weather patterns, congestion analysis, ML experiments, and dashboard reporting.

Out of scope

Live feeds, GPS route guidance, production deployment, and traffic-control actions.

Literature Review and Existing System

Author / year	Topic or method in presentation	Limitation noted
Zhu et al. (2019)	Big-data analytics in intelligent transport	Workflow integration
Nguyen et al. (2022)	Big-data traffic management	Conceptual analysis
Amini et al. (2017)	Distributed traffic-control analytics	Limited prediction
Maarala et al. (2015)	Spark traffic-stream analytics	Streaming emphasis
Jankovic et al. (2021)	ML traffic-flow prediction	Needs labelled history

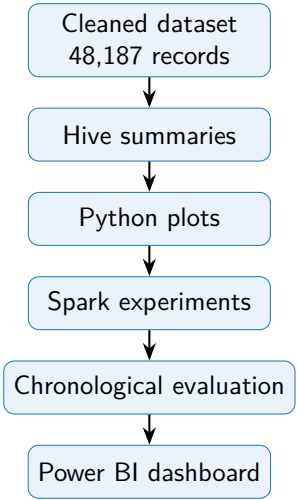
Existing-system limitations

Prior work may study processing, prediction, or analytics separately. Leakage can inflate a model score.

Project research gap

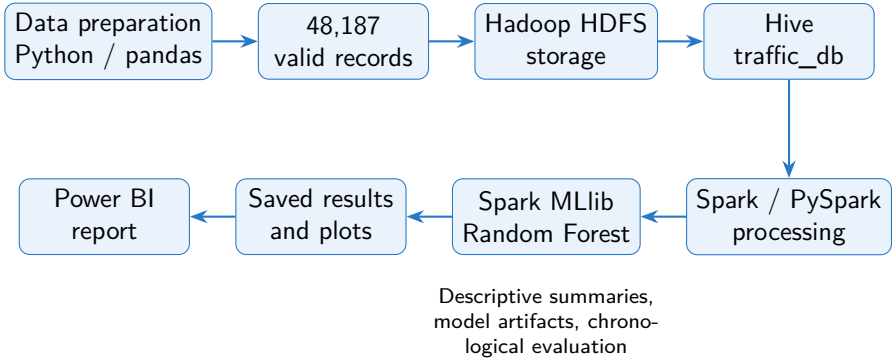
TrafficSense links historical preparation, Hadoop/Hive, Spark experiments, and dashboard reporting; no scientific novelty is claimed.

Proposed System and Key Contributions



- Cleaned analytical dataset with 48,187 valid records.
- Hive summaries for congestion, hourly traffic, and weather.
- Python visualization scripts and stored result artifacts.
- Spark Random Forest experimentation, including a separate next-hour task.
- Chronological test evaluation of the next-hour experiment.
- Power BI dashboard with historical and predicted-label distributions.
- Actual-versus-predicted distribution comparison.

TrafficSense: Big Data System Architecture



Architecture note

The diagram represents the logical workflow; it does not claim a verified multi-node cluster.

Dataset: Metro Interstate Traffic Volume

Record counts

- Original: 48,204 reported records
- Valid analytical dataset: **48,187 retained records**

Source / format

Kaggle dataset; CSV.
`https://www.kaggle.com/datasets/pooriamst/metro-interstate-traffic-volume`
Date range: 2012–2018

Feature group	Fields
Temporal	date_time, year, month, day, hour, day_of_week
Weather	temp, rain_1h, snow_1h, clouds_all, weather_main, weather_description
Context	holiday
Traffic	traffic_volume
Derived target	congestion

Target and geographic limits

Categories derive from traffic-volume thresholds (exact values unavailable here). The next-hour task predicts the next hourly category. The dataset lacks road-network/GPS detail for route-specific predictions.

Data Cleaning, Feature Engineering, and Exploratory Analysis

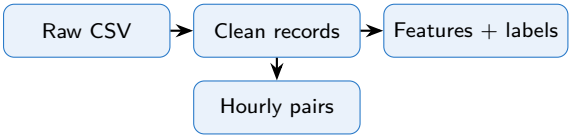
1. Start with 48,204 reported CSV records.
2. Filter the accidental header; retain 48,187 valid rows.
3. Parse time and derive calendar/weather features.
4. Derive congestion labels and prepare tables.

Schema example

Field	Role
date_time	Observation time
traffic_volume	Measured volume
weather_main	Weather category
hour	Derived time feature
congestion	Derived label

Next-hour pairs

Sort by time; keep pairs exactly one hour apart; split chronologically. The 17-row difference is not confirmed duplicate removal.



Methodology and Algorithm

1. Ingest historical dataset.
2. Clean and prepare records.
3. Store and query with HDFS / Hive.
4. Run Spark descriptive analytics.
5. Construct consecutive-hour targets.
6. Prepare features and split.
7. Train Spark MLlib Random Forest.
8. Evaluate and visualize results.

Prediction Input Variables: current traffic_volume, temp, rain_1h, snow_1h, clouds_all, hour, day_of_week, month, and weather_index.

Historical held-out result only, not a live metric.

Model configuration

- Random Forest ensemble
- 50 trees, max depth: 10
- Seed: 42

Recorded result

93.18%

chronological held-out test accuracy

Technology Stack

Storage and Big Data

- Hadoop 3.3.6 / HDFS — file storage.
- Hive 3.1.3 — SQL-like queries and table metadata.

Programming and preparation

- Python, pandas — data handling scripts.
- Matplotlib — chart generation.
- Jupyter Notebook — exploratory analysis.

Processing and ML

- Spark / PySpark 3.5.1 — data processing and experiments.
- Spark MLlib — Random Forest training and evaluation.

Reporting and development

- Microsoft Power BI — interactive dashboard.
- Ubuntu terminal, VS Code — environment.

Implementation: Project Structure and Core Code

Project structure

```
TrafficSense_Project/
|-- 01_Dataset/
|   |-- Metro_Interstate_Traffic_Volume.csv
|   `-- traffic_cleaned.csv
|-- 02_Source_Code/
|   |-- first_stage.ipynb
|   |-- spark_hive_analysis.py
|   |-- create_visualizations.py
|   |-- train_next_hour_model.py
|   `-- README.md
|-- 03_Hive_SQL/
|   |-- analysis_reference.sql
|   `-- README.md
|-- 04_Results/
|   |-- congestion_distribution.png
|   |-- feature_importance.png
|   |-- feature_importance_without_volume.csv
|   |-- next_hour_forecast_results.csv
|   |-- traffic_by_hour.png
|   `-- weather_impact.png
|-- 05_Dashboard/hello.pbix
|-- 06_Documentation/
|   |-- artifact_checklist.md
|   `-- presentation_notes.md
|-- README.md
`-- README_backup.md
```

Hive analytical query

```
abhijeet@LAPTOP-VBFS4NSG:~$ hive -e "SELECT c
ongestion, COUNT(*) AS record_count FROM traf
fic_db.traffic_data WHERE congestion IN ('Low
','Medium','High') GROUP BY congestion ORDER
BY congestion;"
Total MapReduce CPU Time Spent: 0 msec
OK
High    11353
Low     15216
Medium  21618
Time taken: 17.49 seconds, Fetched: 3 row(s)
```

HDFS snippet

```
abhijeet@LAPTOP-VBFS4NSG:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [LAPTOP-VBFS4NSG]
abhijeet@LAPTOP-VBFS4NSG:~$ jps
1204 Jps
709 DataNode
921 SecondaryNameNode
604 NameNode
```

Testing, Validation, and Integrity Checks

Test	Expected result	Recorded outcome
Dataset cleaning	Filter accidental header	48,187 valid records retained
Hive aggregation	Count labels	High 11,353; Low 15,216; Medium 21,618
Hourly analysis	Aggregate by hour	Hourly averages available
Weather analysis	Aggregate by condition	Summary available
Next-hour preparation	Keep 1-hour pairs	37,986 examples
Chronological split	Preserve time order	30,388 train; 7,598 test
Model evaluation	Test held-out data	93.18% historical accuracy
Dashboard	Show summaries and labels	Existing project visualization PBIX

Validation strategy

Use a time-ordered holdout; keep future observations out of training and separate test metrics from dashboard counts.

Integrity checks

Distribution bars are not a confusion matrix; precision, recall, and F1 values were not supplied.

Results: Historical Traffic Patterns and ML Experiment

Model result

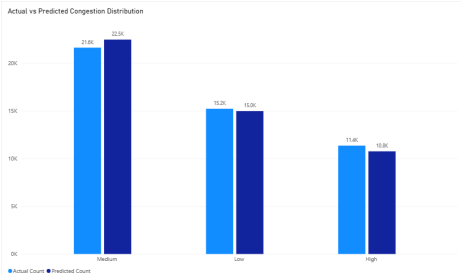
93.18%

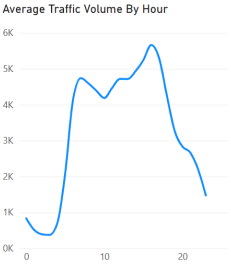
chronological test accuracy

Split

- 30,388 train
- 7,598 test
- Split: 2017-11-16 20:00

Model: Random Forest, 50 trees, depth 10.



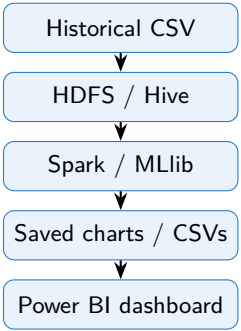


Historical data snapshot

48,187 records; average volume ≈ 3.26K; peak ≈ 7K. Hourly averages: 15: 5,240.46; 16: 5,663.76; 17: 5,310.08. Category averages: High 5,751.82; Medium 3,685.86; Low 794.55.

Working Demonstration: From Data to Dashboard

1. Initialize Hadoop/HDFS: start services; verify availability.
2. Verify artifacts: check recorded paths; do not assume availability.
3. Demonstrate Hive: open `traffic_db`; run the count query if connected.
4. Show Spark/ML: present saved results and the chronological split.
5. Show Power BI: review KPIs, hourly, weather, and label views.



Fallback

If services fail, present saved charts, CSVs, or the dashboard PBIX as saved project outputs.

Team Contributions

Abhijeet Rameshkumar Nair
CEC24AD005

Primary technical implementation, system architecture, and project integration.
Support: dataset preprocessing and feature engineering.

Ananthukrishna CU
CEC24AD014

Literature survey and dataset selection support.
Support: presentation preparation and documentation.

Abhinav KP
LCEC24AD058

Visualization, dashboard development, and analysis support.
Support: testing assistance.

Amarnath A
LCEC24AD061

Testing support and documentation.
Support: project review assistance.

Team collaboration included dataset selection, architecture reviews, project integration, and presentation milestones. Contributions are represented by the roles in the progress presentation; this does not imply identical coding responsibilities.

Conclusion, Limitations, and Future Scope

Achievements

TrafficSense demonstrates a historical-data workflow across preparation, Hadoop/Hive, Spark/ML, and Power BI. The project produced hourly and weather summaries, a dashboard of label distributions, and a separate next-hour Random Forest experiment with 93.18% chronological test accuracy.

Limitations

- Historical observations only; no live feeds.
- Limited road-network and GPS detail.
- 93.18% historical accuracy is not live performance.

Future scope

- Evaluate precision, recall, F1, and confusion matrix.
- Compare with suitable baseline models.
- Add broader data and geographic information.
- Explore sensors/GPS, Kafka, or streaming; deploy only after validation.

References & Questions

1. Metro Interstate Traffic Volume dataset. Kaggle. `https://www.kaggle.com/datasets/pooriamst/metro-interstate-traffic-volume`
2. Apache Hadoop official documentation.
3. Apache Hive official documentation.
4. Apache Spark official documentation, including MLlib.
5. Microsoft Power BI official documentation.
6. Breiman, L. (2001). "Random Forests." *Machine Learning*, 45, 5–32.
7. Zhu et al. (2019); Nguyen et al. (2022); Amini et al. (2017); Maarala et al. (2015); Jankovic et al. (2021). Literature-review entries from the earlier project presentation; metadata to verify.

Thank You

Questions and Discussion

TrafficSense